



3차 과제 투두리스트 코드를 유지하면서, 4차 과제 필수 조건인 이중 리스트 구조와 try-except 예외 처리를 추가하고 싶어. 기존 main.py를 어떻게 백업하고 새로 짜야 할까?

3차 과제 코드를 다치지 않게 안전하게 따로 빼두고, 4차 과제 조건(이중 리스트 + 예외 처리)에 맞춰 깔끔하게 리팩토링하는 전체 프로세스입니다.

기존 백업 방법부터 새롭게 적용할 코드의 구조적 차이점까지 한눈에 보기 좋게 정리해 드릴게요.



1. 기존 main.py 안전하게 백업하기

새로운 코드를 짜기 전에, 기존에 잘 작동하던 3차 과제 코드가 날아가지 않도록 VS Code에서 먼저 백업 파일을 만들어야 합니다.

1. 파일 복사 및 이름 변경:

- VS Code 왼쪽 파일 탐색기에서 main.py 를 클릭한 뒤 `Ctrl + C` → `Ctrl + V` 를 누릅니다.
- 복사되어 새로 생긴 파일의 이름을 ** 3차과제.py **로 명확하게 변경합니다.

2. main.py 비우기:

- 원래 있던 main.py 파일을 열고 전체 선택(`Ctrl + A`) 후 삭제(`Delete`)하여 빈 파일로 만듭니다.
- 이 빈 main.py 공간에 이제 4차 과제 필수 조건이 반영된 새 코드를 채워 넣을 것입니다.



2. 4차 과제 필수 조건 반영하여 새로 짜기 (main.py)

4차 과제는 기존의 단일 딕셔너리나 리스트 구조에서 발전하여, 리스트 안에 또 리스트가 들어가는 '이중 리스트(2D List)' 구조를 활용해야 하며, 사용자의 잘못된 입력에 대응하는 try-except 예외 처리가 핵심입니다.

아래 구조를 그대로 빈 main.py 에 적용하시면 됩니다.

python

```
# -*- coding: utf-8 -*-
# 파일이름 : 60251781 옥민경 파이썬 4차과제
# 작성자 : 옥민경

# [필수조건 1] 이중 리스트(2D List) 구조 정의 (ID, 할일내용, 상태, 우선순위)
todo_list = [
    [1, "파이썬 4차 과제 제출하기", "미완료", 5],
    [2, "깃허브 리포지토리 커밋 확인", "미완료", 3]
]
current_id = 3 # 다음 추가될 항목의 고유 ID
```

de

+ Gemini에게 물어보세요



```
print("\n--- [새로운 할 일 추가] ---")
task = input("추가할 할 일을 입력하세요: ").strip()
```



While True 무한 루프를 돌려서 프로그램을 만드는데, 사용자가 4번을 눌러서 종료하기 전까지 데이터가 안전하게 유지되다가 종료 시점에 메모리가 깔끔하게 해제되는 구조가 맞는지 로직 검증해 줘.

민경님이 구상하신 `while True` 무한 루프 내에서 데이터를 유지하다가 종료 시점에 프로그램을 빠져나오는 구조는 파이썬에서 대화형 콘솔 프로그램을 만들 때 사용하는 가장 표준적이고 안전한 정석 로직이 맞습니다!

이 구조가 왜 안전하고 메모리 관리가 어떻게 깔끔하게 이루어지는지 구조적 관점에서 명쾌하게 검증해 드릴게요.

1. 데이터가 종료 전까지 안전하게 유지되는 이유 (전역 변수 스코프)

현재 코드 구조를 보면 `todo_list` 와 `current_id` 같은 핵심 데이터가 특정 함수 내부가 아닌, **파일의 가장 최상단(전역 스코프, Global Scope)**에 선언되어 있습니다.

- 데이터의 수명(Lifetime): 파이썬에서 전역 공간에 선언된 변수는 프로그램이 시작되는 순간 메모리에 로드되어, 프로그램이 완전히 종료될 때까지 절대 먼저 사라지지 않고 계속 살아있습니다.
- 무한 루프와의 시너지: `while True` 루프가 몇 백 번을 돌든, 사용자가 1번을 눌러 함수를 들어라날락하든 상관없이 `todo_list` 주소 값은 고정된 채 데이터가 덮어써 지거나 초기화되지 않고 안전하게 유지됩니다.

2. 종료 시점에 메모리가 깔끔하게 해제되는 원리

사용자가 4번 을 눌러 `break` 를 만나는 순간, 파이썬 내부에서는 다음과 같은 순서로 자동 청소(Garbage Collection)가 일어납니다.

[사용자 4번 입력] ➡ `break` 호출 ➡ `while` 무한루프 탈출 ➡ `main()` 함수 종료 ➡ 프로그램 프로세스

1. `break` 를 통한 안전한 루프 탈출: 무한 루프를 강제로 강제 종료(`Ctrl + C` 등)하는 것이 아니라, `break` 명령어를 통해 루프를 정상적으로 빠져나옵니다.
2. 참조 카운트(Reference Count)의 제로화: 프로그램이 종료되면 전역 공간에 있던 `todo_list` 를 참조하는 동력이 상실됩니다. 파이썬의 메모리 관리 시스템(가비지 컬렉터)은 참조가 없어진 데이터들을 메모리에서 완전히 지워버립니다.
3. OS 레벨의 자원 회수: 최종적으로 파이썬 프로세스가 닫히면서, 이 프로그램이 노트북 RAM(메모리)에 차지하고 있던 구역을 운영체제(OS)가 통째로 회수해 가기 때문에 메모리 누수(Memory Leak) 없이 100% 깔끔하게 청소됩니다.

3. 완벽한 마무리를 위한 로직 체크포인트 (with open)

민경님 코드는 `with open('data.txt', 'a') as f:` 로 파일 시스템을 안전하게 관리하고 있습니다.

+ Gemini에게 물어보세요



- 가비지 컬렉션은 프로그램이 종료될 때 자동으로 수행되지만, 종료 시점까지 메모리가 정리되지 않거나, 이 끝나

도 파일 시스템이 메모리를 붙잡고 있는 버그가 생길 수 있습니다.



파이썬에서 전역 공간에 선언된 변수는 프로그램이 시작되는 순간 프로그램이 완전히 종료될 때까지 절대 먼저 사라지지 않고 계속 살아있습니다.

- 무한 루프와의 시너지: `while True` 루프가 몇 백 번을 돌든, 사용자가 1번을 눌러 함수를 들어락날락하든 상관 없이 `todo_list` 주소 값은 고정된 채 데이터가 덮어써 지거나 초기화되지 않고 안전하게 유지됩니다.

2. 종료 시점에 메모리가 깔끔하게 해제되는 원리

사용자가 4번 을 눌러 `break` 를 만나는 순간, 파이썬 내부에서는 다음과 같은 순서로 자동 청소(Garbage Collection)가 일어납니다.

[사용자 4번 입력] ➡ `break` 호출 ➡ `while` 무한루프 탈출 ➡ `main()` 함수 종료 ➡ 프로그램 프로세스

- `break` 를 통한 안전한 루프 탈출: 무한 루프를 강제로 강제 종료(`Ctrl + C` 등)하는 것이 아니라, `break` 명령어를 통해 루프를 정상적으로 빠져나옵니다.
- 참조 카운트(Reference Count)의 제로화: 프로그램이 종료되면 전역 공간에 있던 `todo_list` 를 참조하는 동력이 상실됩니다. 파이썬의 메모리 관리 시스템(가비지 컬렉터)은 참조가 없어진 데이터들을 메모리에서 완전히 지워버립니다.
- OS 레벨의 자원 회수: 최종적으로 파이썬 프로세스가 닫히면서, 이 프로그램이 노트북 RAM(메모리)에 차지하고 있던 구역을 운영체제(OS)가 통째로 회수해 가기 때문에 메모리 누수(Memory Leak) 없이 100% 깔끔하게 청소됩니다.

3. 완벽한 마무리를 위한 로직 체크포인트 (with open)

민경님 코드에서 메모리와 자원을 아끼는 가장 결정적인 부분은 바로 `with open()` 구문입니다.

- 기존 방식의 위험성: 옛날 방식인 `f = open()` 을 쓰고 깜빡하고 `f.close()` 를 안 적으면, 프로그램이 끝나도 파일 시스템이 메모리를 붙잡고 있는 버그가 생길 수 있습니다.
- `with` 구문의 장점: `with open(...) as f:` 방식을 사용하면, 내부 블록 코드가 끝나거나 심지어 저장 도중 에러(`except`)가 발생하더라도 파이썬이 알아서 문을 닫고(`close`) 관련 메모리를 즉시 해제해 줍니다.

결론: > 현재 작성된 구조는 전역 변수를 통한 데이터 유지, `break` 를 통한 프로세스 정상 종료, `with open` 을 통한 파일 자원 자동 해제까지 삼박자가 모두 맞물려 있어 메모리 관리나 로직 측면에서 한 치의 오차도 없는 완벽한 구조입니다. 안심하고 사용하셔도 좋습니다!



Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. [개인정보보호 및 Gemini](#)

+ Gemini에게 물어보세요





이런 오류가 났는데 어떻게 해결해야할까

마지막 사진(`reportUndefinedVariable` 오류 화면)을 보니, 컴퓨터가 왜 화가 났는지 딱 보입니다!

지금 `for item in todos:` 구문이 `show_todos` 함수 내부가 아니라 완전 바깥으로 빠져나와 있어서 발생하는 오류예요. 파이썬은 들여쓰기(스페이스바 4칸)로 구역을 나누는데, 줄이 안 맞다 보니 함수 밖에 있는 `todos` 가 뭔지 못 찾겠다고 징징대는 중입니다!

직접 타이핑하지 않고 키보드 단축키 딱 두 번으로 들여쓰기 줄 맞춰서 에러를 없애는 가장 빠른 방법을 알려드릴게요.

🔧 단축키로 들여쓰기(Indentation) 오류 해결하기

1. 마우스로 드래그하기: 오류가 난 43번째 줄(`for item in todos:`)부터 54번째 줄(`print(row_str)`)까지 마우스로 속 드래그해서 블록을 지정합니다.
2. Tab 키 누르기: 그 상태에서 키보드의 Tab 키를 딱 한 번만 누릅니다. (그러면 선택한 모든 줄이 오른쪽으로 스페이스 4칸만큼 한 방에 예쁘게 이동합니다.)
3. 확인하기:
`for` 문이 바로 위 `print("_"*65)` 라인과 수직으로 시작 위치가 똑같아졌는지 확인합니다.

줄이 딱 맞는 순간 아래쪽에 드던 파란색 에러 창("`todos`"이 정의되지 않았습니다.)이 마법처럼 싹 사라질 거예요!

💡 하나 더! 40번째 줄 표 제목 오타도 같이 고쳐주세요

지금 40번째 줄 출력 양식을 보면 중괄호 안이 `{ 'ID':<5> }` 가 아니라 `{ 'ID':5 }` 이런 식으로 콜론(:) 뒤에 화살표가 빠져 있어서 나중에 표가 깨질 수 있습니다.

줄 맞추는 김에 40번째 줄을 아래 문장으로 똑같이 고쳐주시면 나중에 실행했을 때 표가 빠тол어지지 않고 완벽하게 나옵니다!

python

```
print(f"{'ID':<5> | {'할 일 내용':<30> | {'상태':<10> | {'우선순위':<8>})
```

얼른 드래그해서 Tab 키 한 번 누르고 오류 창 없애버리시다! 바로 해결될 거예요.



+ Gemini에게 물어보세요

